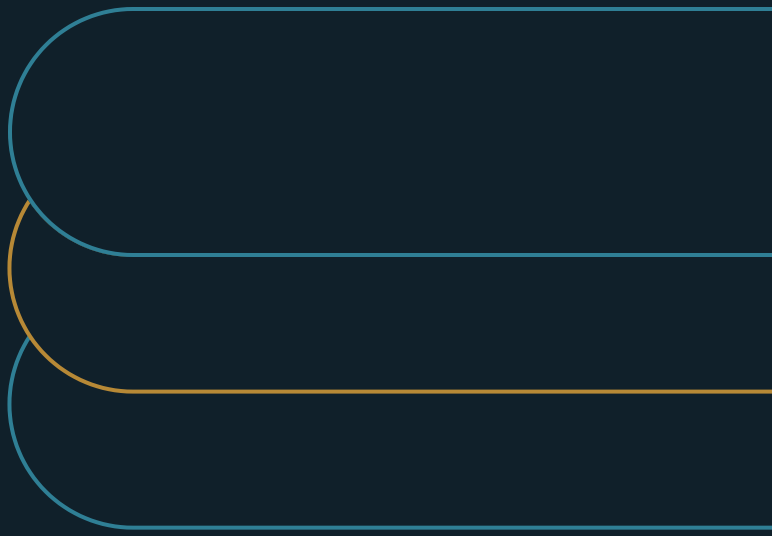
11

WHITE PAPER · WORK, ADOPTION & JUDGMENT

The Exception Path Is the Real Automated Workflow

Design recovery before declaring a workflow automated



Marco Policani

Enterprise Portfolio · PMO · AI Operating Governance

policani.net · linkedin.com/in/marcpolicani

July 2026

EXECUTIVE SUMMARY

A workflow is automated when its exception path is designed, staffed, and rehearsed - not when its happy path is fast.

Automating the routine cases means the cases left for humans are, by construction, the hardest ones. The exception stream is where judgment, compliance exposure, and customer trust concentrate - and where most automation programs stop designing. This paper sets out the six-part exception path a workflow must have before 'automated' is an honest description.

01 Replace the manual-review bucket with a taxonomy of why processing stopped. The single bucket destroys the signal.

02 Assign two owners per class: one resolves the case, one fixes the condition that keeps producing cases like it.

03 Measure causes retired, not volume suppressed. A class that vanished because a threshold moved is a risk transfer.

The argument

The happy path proves throughput. The exception path proves whether the operation can detect what falls outside the rules, route it to an owner, limit the damage, keep the evidence, recover, and learn.

Whenever a team tells me their workflow is automated, I ask to see the exception queue. The reaction usually answers the question. The pattern I keep finding is a companion inbox — often literally a shared mailbox — where everything the automation could not handle quietly piles up. No classification. No owners. No service expectation. No record of what was tried. A few people work it part-time, in arrival order, while the oldest items age into real risk. The automation processes most of its volume beautifully. The operation's actual risk, cost, and reputation live in the rest.

That distribution is the rule, not a fluke, and it follows from what automation is. Moving routine cases out of human hands means the cases left in human hands are — by construction — the ambiguous, the malformed, the conflicting, the novel, and the unsafe. The exception stream is where the hardest judgment concentrates, where compliance exposure hides, and where customer trust is actually won or lost. Yet automation proposals describe it in one sentence ("exceptions route to manual review"), staff it as an afterthought, and measure it not at all. That residual is where human oversight is weakest: monitoring automation is itself an error-prone task, and vigilance and error-detection degrade the more reliably the automation runs [3]. This is the operating form of what Lisanne Bainbridge called the “ironies of automation”: the better the machine handles the routine, the harder and rarer the cases it hands back to people, and the less practiced those people are at meeting them. The exception path is where that irony is either designed for or discovered at the worst possible time.

This paper inverts the design order. The exception path is the real workflow. The happy path is the optimization. It sets out a six-part path — classify, detect and contain, route with context, own the response, recover to a known state, learn and redesign — as the standard a workflow must meet before "automated" is an honest description. NIST's risk-management framework supplies the governing frame, and Microsoft's red-team experience shows that failure-finding must be deliberate and resourced [1][2]. The six parts are what years of opening other people's exception queues taught me to require. The paper is written for workflow owners, automation leaders, AI program owners, operations executives, and risk teams.

The economics of the other nine percent

Why does the exception path decide the economics? Because its costs are expensive per case and its failures are consequential per case. A routine case costs seconds of compute. An exception costs specialist minutes or hours, plus the routing tour to find the specialist, plus rebuilding the context once it arrives. A routine failure is a retry. A mishandled exception is a lapsed obligation, a wrong payment, an angry customer, or a regulator's letter. Small share of volume, large share of consequence — a structural asymmetry.

And it compounds as automation succeeds. Raise the happy path from ninety to ninety-six percent and the queue shrinks — but the cases still in it are now the hardest six percent, the ones the improved model still could not touch. Every gain in coverage distills the residue. An operation that staffs its exception handling for last year's mix will find this year's mix harder per case. "The queue is smaller now" and "the queue is fine now" are different claims. Only the second one needs evidence.

There is also a visibility trap in the accounting. Happy-path metrics are automatic: counts, speeds, rates, all free from the pipeline. Exception metrics require someone to decide the queue is worth instrumenting. So the dashboard fills with the workflow's best news while its risk concentrates where the instruments are not. Nobody decided to hide the queue. Nobody decided to look.

The six-part exception path

The six parts run in operating order: what the case is, how it is caught and held safe, how it reaches the right person, who answers for it, how the world gets restored, and how the system gets better. Each part has a characteristic failure and a working design. Together they are the difference between an exception path and an exception pile.

EXHIBIT 1

The exception-recovery path

01

Classify and contain

Name why processing stopped, size urgency, and stop consequence from spreading.

02

Route with context

A decision package, not an alarm. The reviewer acts without hunting.

03

Assign two owners

One resolves the case; another removes the condition that created it.

04

Recover to a known state

Every touched system corrected. Partial actions unwound.

05

Learn and redesign

Read the queue quarterly. Retire causes, not volume.

Classify exceptions

The first part replaces the generic manual-review bucket with a taxonomy: the reasons ordinary processing was unsafe or impossible. Missing or conflicting data. Low confidence. Policy conflict. A failed dependency. An unusual or unauthorized request. Suspected harm. And the genuinely novel case. Each class implies a different urgency, a different skill, and a different fix at the source. That is exactly the information the single bucket destroys.

Classification converts the queue from a pile into a signal. Frequency by class shows where the automation's envelope actually ends. Severity by class shows where the risk lives. Recurrence by class shows which upstream fix would pay. The taxonomy also drives staffing — a policy-conflict class needs someone with policy authority; a data-conflict class needs someone who can bind the source systems — and it separates the routine-recoverable from the contain-it-now cases, which no first-in-first-out mailbox can do.

Build the taxonomy from history, not imagination. Sample the last few hundred exceptions from the queue as it actually exists. Have the people who resolved them name the reasons in their own words. Expect eight to twelve classes to cover nearly everything. The leftover "novel" class is the taxonomy's most important member, not a gap in it: novel cases are where tomorrow's classes announce themselves.

The taxonomy also protects against a subtle staffing error: treating exception volume as one number. A hundred weekly exceptions that are ninety percent routine data problems need a clerk-hour profile. The

same hundred with the proportions reversed need policy authority on call. Classification makes the difference visible before the staffing model discovers it through backlog.

Detect and contain

The second part is the system's obligation to know when it is out of its depth — and to stop moving when it is. Detection is boundary awareness: input validation, confidence thresholds, policy checks, dependency health, anomaly signals. Containment is the safe state that follows: hold the case, release nothing partial, mark what was attempted, and surface the event while response options are still open.

The failure this prevents is the plausible guess — the system that, instead of declaring an exception, proceeds with its own best interpretation of a malformed input or a silently failed dependency. Plausible guesses are the most expensive failure class in automation. They convert detectable exceptions into undetected errors, which surface later, downstream, stripped of the context that would have made them cheap to fix. A workflow that never raises exceptions has not earned the word robust; more likely it is lucky, or quietly guessing, and the difference is discovered at audit. Generative systems make this failure more likely, not less: they are prone to fluent, confident output that is unsupported or fabricated, which is precisely the plausible guess that slips past a hurried check [4].

Partial completion deserves its own design attention. An automation that runs three steps of five before failing has created a state no one designed. Containment means all-or-nothing behavior, or explicit checkpoints with known recovery steps. The proof that this part works is measurable. Detection rates on seeded bad inputs. Error rates at the boundary. Time from boundary event to contained state. And the count of partial actions found later — a number whose target is zero, and whose actual value is the honesty test.

Route with context

The third part delivers the case to a human as a decision package, not an alarm. The reviewer needs what the system knew: the input, what was attempted, why processing stopped, the affected records and parties, and the clock on the case — assembled at handoff, not reconstructed across four systems and two chat threads. Context reconstruction is the dominant cost in most exception handling, and it is pure waste. The system had the context and dropped it.

Routing follows the taxonomy: each class maps to a role with the authority and skill to resolve it, with a fallback and an escalation clock. The anti-pattern is broadcast routing — the exception channel everyone can see and no one owns. It produces the bystander queue: visible to twelve people, worked by none, aging politely until someone important asks about a specific case.

The measures are queue mechanics. Could the reviewer act without hunting? How long until acceptance? How many clarification round-trips? How many misroutes? And queue age by class — the executive view. A routine-data class aging a week is an efficiency problem. A suspected-harm class aging a day is an incident in progress.

Respect the reviewer's clock explicitly. Every routed case carries two times: how long it may wait for acceptance, and how long resolution may take once accepted. Both set by class consequence, both

visible, both escalating automatically when breached. The clocks convert "someone will get to it" into a managed commitment.

Assign response ownership

The fourth part separates two jobs the mailbox blurs into one. Resolving this case. And fixing the condition that keeps producing cases like it. Case ownership belongs in the operation: someone answers for each item reaching resolution on time. Condition ownership belongs wherever the cause lives. The product team whose parser drops the field. The data owner whose feed conflicts. The policy owner whose rule contradicts practice.

Without the second owner, the operation clears cases forever. The queue becomes a permanent institution with headcount, dashboards, and no mandate to shrink. The same hundred-case-a-week class gets resolved heroically, forever, while its cause sits untouched. The design is a standing loop. Recurrence reports by class flow to condition owners. Fix commitments come back with dates. And classes that outlive their commitments escalate as risks someone must sign for.

The signature matters. An exception class that recurs because fixing it is not worth the cost is a legitimate business decision — priced, owned, revisited. The same class recurring because no one with authority ever looked is an unpriced liability that happens to sit in an operations queue. The ownership design exists to make the two distinguishable.

Recover to a known state

The fifth part defines what "resolved" means, and it means more than "ticket closed." Recovery restores the world. Data corrected in every system the case touched. Partial transactions unwound or completed. Downstream teams told where they used wrong outputs. Customer commitments repaired. The workflow returned to a state where ordinary processing can safely resume. A case closed by workaround — fixed in one system, left wrong in two others — is not resolved. It is deferred, with interest.

Recovery steps are per-class and rehearsed. The rollback for a partial payment run. The reconciliation for a duplicate-record class. The re-entry check before a contained case rejoins the flow. Rehearsal is what separates a procedure from a document. The first live run of an unrehearsed rollback is itself an incident, performed under pressure, on real data. The evidence: recovery time by class, reconciliation results, repeat-contact rates (a customer calling again is recovery failing in public), and leftover errors found by cross-system sampling.

The re-entry check closes the loop on containment. Before a resolved case rejoins ordinary processing, something verifies that the condition which ejected it is actually fixed. Cases that bounce — exception, resolution, re-entry, same exception — are the signature of workaround-as-resolution. Their rate belongs on the same page as the recovery times.

Evidence preservation runs through recovery and beyond it: what the system attempted, what the reviewer decided, and why, kept with the case. The record serves three masters at once. The audit that will eventually ask. The recurrence analysis that needs resolved cases as raw material. And the next reviewer of a similar case, who inherits reasoning instead of reinventing it.

Learn and redesign

The sixth part turns the exception stream into the workflow's improvement budget. Every class is a bet about what to fix. Validation that would retire a bad-data class. A source-system change that would end a conflict class. A policy clarification that would dissolve a policy-conflict class. An envelope expansion the evidence now supports — or a contraction the evidence now demands. The queue, read quarterly, is the most honest product-management document the automation has.

This part's characteristic corruption is threshold gaming. Manual-review volume becomes a target, and the cheap way to hit it is raising confidence thresholds or reclassifying exceptions as acceptable outputs — declaring victory by redefining defeat. The defense is measuring causes retired rather than volume suppressed. A class that disappeared because validation now catches it upstream is a win. A class that disappeared because the threshold moved is a risk transfer to whoever meets the errors downstream.

Learning also feeds the test machinery. Resolved exceptions are pre-labeled hard cases, and the interesting ones belong in the workflow's regression set, so the next model or prompt change gets tested against the hard cases the operation has already paid for. An automation program that discards its exception history is throwing away the only test data that arrives pre-validated by consequence. Machine-learning systems in particular accrue hidden maintenance debt as data and the outside world shift, so a rising exception stream is also an early signal that the automation itself is drifting [5].

AI-based automation widens the taxonomy in one specific way. The low-confidence and suspected-harm classes stop being rare. A deterministic parser fails on malformed input. A generative system can also fail on well-formed input it merely handles badly, with output that looks complete. Confidence thresholds, sampling of accepted outputs, and downstream verification become detection instruments in their own right, and the reviewer's job shifts toward evaluation [2]. The six parts hold unchanged.

The path as the readiness standard

Run the six parts together and the opening inversion becomes operational: a workflow is automated when its exception path is designed, staffed, instrumented, and rehearsed — not when its happy path demonstrates throughput. The readiness review is short and concrete. Show the taxonomy. Show containment behavior on seeded bad cases. Show a routed case's context package. Name the case and condition owners. Walk the rehearsed recovery for the worst class. Show last quarter's retired causes. A team that can do those six things has an automated workflow. A team that cannot has a demo with an inbox.

The standard scales the right way, too. A low-consequence workflow earns a light path: few classes, simple containment, one owner, modest instrumentation. A workflow touching money, customers, or regulated records earns the full design. What no workflow earns is the sentence "exceptions route to manual review" standing in for all six parts — because that sentence, unexamined, is where the shared mailbox comes from.

Building the path for a live workflow

The build order follows the evidence:

- Build the taxonomy from real historical cases and frontline interviews.
- Design detection, containment, routing, ownership, evidence, and recovery for each high-consequence class.
- Exercise the path with missing, ambiguous, conflicting, and unsafe inputs before launch.
- Review exception demand as a capacity, product, and operations signal.

Pre-launch exercising is where the path proves it exists. Feed the workflow its monsters deliberately — missing fields, conflicting records, ambiguous categories, an unauthorized request, an unsafe output — and watch each one classify, contain, route, and recover end to end. The exercise costs a day and finds the gap between the path as drawn and the path as built. Every first exercise finds one. A launch gate that requires the exercise report has, in one requirement, made "exceptions route to manual review" an impossible sentence to ship.

For workflows already live — which is most of them — start with archaeology. Open the existing queue, classify a sample, and triage by severity before designing anything. In my experience, the first afternoon of classification tends to surface the items that could not wait: the aged compliance notices, the dispute quietly becoming a complaint. Expect the initial classification to reprice the whole program's risk posture, usually upward.

Volume forecasting for the path belongs in every scale decision, and the arithmetic is unforgiving. Doubling happy-path volume roughly doubles exception volume at the current envelope, and the specialist pool does not double by wishing. A scale proposal should state expected exception load by class, the staffing that absorbs it, and the retirement work that will narrow it — or concede that the plan is to let the queue age. Making that concession explicit has stopped more premature scale-ups than any model metric.

Staff the path as a designed role, not a punishment posting. Exception handling done well is senior work: judgment on the hardest cases, pattern recognition across them, authority to bind systems and policies. Organizations that treat it as overflow labor get the queue quality they staffed for. The quarterly exception review, chaired by the workflow owner with condition owners present, is where the role's findings become the program's roadmap.

What the record should show

A working path leaves evidence anyone can inspect:

- A taxonomy with frequency, severity, and recurrence by class — current, not archival.
- Containment results on seeded inputs, and partial-action findings trending to zero.
- Context-complete handoffs and queue age by class, with escalation clocks honored.
- Case and condition owners named per class, with remediation commitments dated.
- Recovery rehearsals completed, and repeat-contact and re-entry-bounce rates.

- Causes retired per quarter — the number that shows the path shrinking itself.

Read the record quarterly with two questions. Where is the path absorbing consequence well? And where is it subsidizing a defect that should be fixed at the source? The first question defends the staffing. The second directs the roadmap. A review that asks only the first builds a better and better buffer around an unimproved machine — competent, expensive, and permanent.

The last number is the health of the whole design. A mature exception path gets quieter over time in the classes it has learned, and louder only at the frontier of new work: new case types, new volumes, new models. A path whose volume is flat across quarters with no retired causes is an operations team subsidizing an unimproved automation, however smoothly the subsidy runs.

The strongest counterargument

Not every rare event deserves bespoke automation. Some exceptions should remain human-owned, and some cases should be excluded entirely. The objective is not zero exceptions but a deliberate path that contains consequence and makes learning possible.

EXHIBIT 2

The readiness review, in six requests

Show the taxonomy
Frequency, severity, and recurrence by class - current, not archival.

Show containment
Behavior on seeded bad inputs, and partial actions trending to zero.

Show a handoff
One routed case's context package, complete at delivery.

Walk the recovery
The rehearsed procedure for the worst class, not the document.

The investment objection — that six-part design is gold-plating for a queue three people were handling fine — should be tested against the queue itself. The test is usually decisive. Classify one sample of the existing pile and price what surfaces. The aged compliance items. The specialist hours lost to rebuilding context. The workaround fixes quietly building reconciliation debt. The repeat contacts. In my experience the sample pays for the design several times over before the design starts. Where it truly does not — low consequence, low volume — the framework itself says so. Earn a light path, and spend the rigor where the queue is expensive.

What 'automated' has to mean

The redefinition is the point. Automated means the failure modes are governed, not merely that the success mode is fast. Design the path for the cases that fall outside the rules — classified by cause, contained before consequence spreads, routed with context intact, owned twice, recovered fully across every touched system, and mined quarterly for the causes worth retiring — and the happy path can then be trusted with everything else. Everything else is what it was actually built for.

The readiness standard travels well, too. It works as a due-diligence question for vendor automations, as an audit lens for inherited workflows, and as a one-line policy for new builds: no launch without the exercised path.

Notes and sources

[1] National Institute of Standards and Technology, "AI Risk Management Framework Core," AI RMF 1.0, 2023. https://airc.nist.gov/airmf-resources/airmf/5-sec-core/

[2] Blake Bullwinkel, Amanda Minnich, Shiven Chawla, Gary Lopez, Martin Pouliot, Whitney Maxwell, Joris de Gruyter, Katherine Pratt, Saphir Qi, Nina Chikanov, Roman Lutz, Raja Sekhar Rao Dheekonda, Bolor-Erdene Jagdagdorj, Eugenia Kim, Justin Song, Keegan Hines, Daniel Jones, Giorgio Severi, Richard Lundeen, Sam Vaughan, Victoria Westerhoff, Pete Bryan, Ram Shankar Siva Kumar, Yonatan Zunger, Chang Kawaguchi, and Mark Russinovich, "Lessons From Red Teaming 100 Generative AI Products," Microsoft Research, January 2025. https://www.microsoft.com/en-us/research/publication/lessons-from-red-teaming-100-generative-ai-products/

[3] Raja Parasuraman and Dietrich H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," Human Factors 52, no. 3 (2010): 381-410. https://depositonce.tu-berlin.de/bitstreams/cafd2873-814b-4c59-bab1-addd42e249d2/download

[4] Ziwei Ji, Nayeon Lee, Rita Frieske, et al., "Survey of Hallucination in Natural Language Generation," ACM Computing Surveys, 2023. https://arxiv.org/abs/2202.03629

[5] D. Sculley, Gary Holt, Daniel Golovin, et al., "Hidden Technical Debt in Machine Learning Systems," Advances in Neural Information Processing Systems 28 (NeurIPS 2015). https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

About the author

Marco Policani is an enterprise portfolio, PMO, and AI operating-governance leader. He builds portfolio governance systems where AI carries the analytical load and named humans own every decision — an approach documented across case studies, walkthroughs, and working governance guides on his portfolio site.

Portfolio & governance library: policani.net/governance · Full portfolio: policani.net · LinkedIn: linkedin.com/in/marcpolicani